

Tema 10: Implementación de listas

1. Estructura interna: Doble enlace y Centinelas

A diferencia del vector, una lista no guarda los elementos juntos en memoria. Cada elemento está en un **Nodo** (o **Item**) independiente que sabe quién tiene delante y quién tiene detrás.

Nodos Centinela (**iteminf** y **itemsup**)

Esta implementación utiliza dos nodos especiales que **siempre existen**, aunque la lista esté vacía: - **iteminf**: El nodo “ficticio” inicial. Su **next** apunta al primer elemento real. - **itemsup**: El nodo “ficticio” final. Su **prev** apunta al último elemento real.

Ventaja: No tenemos que comprobar nunca si un puntero es **nullptr** al hacer inserciones o borrados, simplificando mucho el código. - **Truco para los ejercicios:** Como los centinelas siempre están ahí, el primer elemento real es **iteminf.next** y el último es **itemsup.prev**. Puedes acceder a ellos directamente para hacer operaciones como **swapFirstLast**.

```
template <typename T>
class List {
    struct Item {
        T value;
        Item *next, *prev;
    };
    int _size;
    Item iteminf, itemsup; // Nodos reales (no punteros)
public:
    // ...
};
```

2. El motor: Insertar y Borrar en $\Theta(1)$

La lista es la estructura ideal para insertar/borrar en cualquier punto si ya tenemos la posición. Solo hay que “recoser” los punteros.

Insertar un elemento (**insertItem**)

1. Creamos el nuevo nodo.
2. Lo conectamos con su siguiente y anterior.
3. Actualizamos los punteros de los vecinos para que apunten al nuevo nodo.

```
void insertItem(Item *pitemprev, const T &value) {
    Item *pitem = new Item;
    pitem->value = value;

    pitem->next = pitemprev->next;
    pitem->next->prev = pitem;
    pitem->prev = pitemprev;
    pitemprev->next = pitem;
```

```

    _size++;
}

```

Cómo mover nodos (la regla de los 4 punteros)

En muchos ejercicios (como `moveToEnd` o `moveSecondToLast`), el Juez prohíbe intercambiar los `.value`. Tienes que mover el nodo físicamente: 1. **Desconectar**: Une el vecino anterior con el siguiente (`p->prev->next = p->next` y `p->next->prev = p->prev`). 2. **Reconectar**: Inserta el nodo en la nueva posición ajustando sus nuevos `next` y `prev` y los de sus nuevos vecinos.

Si mover un nodo implica que este sea vecino de sí mismo (casos de 2 elementos), ¡ten cuidado de no perder el puntero antes de tiempo! Se recomienda el uso de punteros temporales.

3. Iteradores: El puente hacia los datos

Como los nodos están dispersos, no podemos usar `[i]`. Usamos **iteradores**, que actúan como un puntero inteligente que sabe cómo moverse por la lista.

- **begin()**: Apunta al primer elemento real (`iteminf.next`).
- **end()**: Apunta al nodo centinela final (`itemsup`). ¡Nunca se debe desreferenciar!

```

class iterator {
    List *plist;
    Item *pitem;
public:
    T& operator*() { return pitem->value; }
    iterator operator++() { // Preincremento (++it)
        pitem = pitem->next;
        return *this;
    }
};

```

Iteradores Circulares: Si un ejercicio pide que la lista sea circular, el `operator++` del último nodo no debe ir a `end()`, sino a `begin()`. En nuestra implementación con centinelas, esto significa saltar de `itemsup` a `iteminf.next`.

::linkedlistviz

4. Gestión de memoria: La Regla de los Tres

Como gestionamos nodos con `new`, debemos ser muy cuidadosos: 1. **Destructor**: Debe borrar TODOS los nodos (usando `removeItem` en bucle). 2. **Constructor de copia**: Debe crear nodos nuevos y copiar los valores. 3. **Operador de asignación**: Limpiar la lista actual y copiar la nueva.

En la asignación `l1 = l2`, siempre debemos comprobar la **auto-asignación** (`this != &l1`) para no borrar los datos que queremos copiar.

Vector vs Lista: ¿Cuándo usar cuál?

Característica	Vector (<code>std::vector</code>)	Lista (<code>std::list</code>)
Acceso aleatorio <code>[i]</code>	$\Theta(1)$	$\Theta(n)$ (hay que recorrer)
Insertar al final	$\mathcal{O}(1)$ amortizado	$\Theta(1)$
Insertar al principio	$\Theta(n)$	$\Theta(1)$
Insertar en el medio (con <code>it</code>)	$\Theta(n)$	$\Theta(1)$

Característica	Vector (<code>std::vector</code>)	Lista (<code>std::list</code>)
Memoria	Bloque contiguo (más rápido por la CPU)	Nodos dispersos (más overhead)

Operaciones con Iteradores

Operación	Código	Explicación
Insertar	<code>it = l.insert(it, val)</code>	Inserta antes de <code>it</code> .
Borrar	<code>it = l.erase(it)</code>	Borra <code>it</code> y retorna el siguiente .
Recorrer	<code>for(auto it=l.begin(); it!=l.end(); ++it)</code>	El patrón estándar.